

# **Proiect Grafică pe Calculator**

Realizarea unei Scene 3D Fotorealiste -  
Campament de Munte pe Matterhorn

Ciurte Mara Luana

Grupa 30232

UTCN

10 ianuarie 2026

# Cuprins

|          |                                           |           |
|----------|-------------------------------------------|-----------|
| <b>1</b> | <b>Introducere</b>                        | <b>5</b>  |
| 1.1      | Scopul proiectului                        | 5         |
| 1.2      | Tehnologii utilizate                      | 5         |
| 1.3      | Structura documentației                   | 5         |
| <b>2</b> | <b>Vizualizarea Scenei</b>                | <b>6</b>  |
| 2.1      | Control cu tastatura și mouse-ul          | 6         |
| 2.1.1    | Control cameră cu tastatura               | 6         |
| 2.1.2    | Control Cameră cu Mouse-ul                | 6         |
| 2.2      | Animații de prezentare                    | 7         |
| 2.2.1    | Sistemul de Keyframe-uri                  | 7         |
| 2.2.2    | Configurarea traseului camerei            | 7         |
| 2.2.3    | Interpolarea fluidă                       | 8         |
| 2.2.4    | Controlul animației                       | 9         |
| 2.3      | Matricele de transformare                 | 9         |
| 2.3.1    | Matricea model                            | 9         |
| 2.3.2    | Matricea view                             | 10        |
| 2.3.3    | Matricea projection                       | 10        |
| 2.4      | Pipeline-ul de transformare complet       | 10        |
| 2.5      | Rezumat funcționalități vizualizare       | 11        |
| <b>3</b> | <b>Sursele de lumină</b>                  | <b>11</b> |
| 3.1      | Lumina direcțională (Soarele)             | 11        |
| 3.1.1    | Inițializare și configurare               | 11        |
| 3.1.2    | Control interactiv al direcției           | 11        |
| 3.1.3    | Ciclul zi/noapte                          | 12        |
| 3.1.4    | Implementare în Shader                    | 12        |
| 3.2      | Lumina punctiformă (focul)                | 13        |
| 3.2.1    | Poziționare și Configurare                | 13        |
| 3.2.2    | Atenuare (Attenuation)                    | 13        |
| 3.2.3    | Efect de Pâlpâire (Flickering)            | 14        |
| 3.2.4    | Iluminare Phong pentru lumina punctiformă | 15        |
| 3.3      | Combinarea surselor de lumină             | 15        |
| 3.4      | Parametri materiale per obiect            | 15        |
| 3.5      | Rezumat Sistem Iluminare                  | 16        |
| <b>4</b> | <b>Moduri de Vizualizare</b>              | <b>16</b> |
| 4.1      | Implementare                              | 16        |
| 4.2      | Descriere Moduri                          | 16        |
| <b>5</b> | <b>Maparea Texturilor și Materiale</b>    | <b>17</b> |
| 5.1      | Încărcarea Texturilor                     | 17        |
| 5.2      | Aplicarea Texturilor                      | 17        |
| 5.3      | Proprietăți Materiale                     | 18        |
| 5.4      | Coordonate UV                             | 18        |

|          |                                              |           |
|----------|----------------------------------------------|-----------|
| <b>6</b> | <b>Generarea Umbrelor</b>                    | <b>18</b> |
| 6.1      | Shadow Map - Configurare                     | 18        |
| 6.2      | Light Space Matrix                           | 19        |
| 6.3      | Shadow Pass - Depth Map Rendering            | 20        |
| 6.3.1    | Depth Map Vertex Shader                      | 20        |
| 6.3.2    | Depth Map Fragment Shader                    | 20        |
| 6.3.3    | Randare Depth Map                            | 20        |
| 6.4      | Scene Rendering Pass                         | 21        |
| 6.4.1    | Light Shader - Vertex                        | 21        |
| 6.4.2    | Shadow Calculation în Fragment Shader        | 21        |
| 6.4.3    | Aplicarea Umbrelor                           | 22        |
| 6.5      | Tehnici Anti-Aliasing pentru Umbre           | 22        |
| 6.5.1    | Shadow Acne Prevention                       | 22        |
| 6.5.2    | PCF (Percentage Closer Filtering)            | 22        |
| 6.6      | Control Umbre                                | 23        |
| 6.7      | Rezumat Tehnic                               | 23        |
| <b>7</b> | <b>Animarea Componentelor</b>                | <b>23</b> |
| 7.1      | Structura modelului animat                   | 23        |
| 7.2      | Puncte pivot pentru rotație                  | 24        |
| 7.3      | Calculul animației cu funcții sinusoidale    | 24        |
| 7.4      | Transformări Ierarhice pentru Aripi          | 24        |
| 7.4.1    | Aripa stângă                                 | 24        |
| 7.4.2    | Aripa dreaptă                                | 25        |
| 7.5      | Randarea corpului                            | 25        |
| 7.6      | Animație în shadow pass                      | 25        |
| 7.7      | Parametri animație                           | 26        |
| 7.8      | Diagrama Transformărilor                     | 26        |
| <b>8</b> | <b>Generarea Dinamică de Obiecte</b>         | <b>26</b> |
| 8.1      | Sistemul Zi/Noapte                           | 26        |
| 8.2      | Încărcarea Dinamică a Sky Dome-ului          | 27        |
| 8.3      | Control Interactiv                           | 27        |
| 8.4      | Actualizarea Culoilor Luminii                | 27        |
| 8.5      | Efecte Vizuale Zi/Noapte                     | 28        |
| 8.5.1    | Ziua                                         | 28        |
| 8.5.2    | Noaptea                                      | 28        |
| <b>9</b> | <b>Detectarea și rezolvarea coliziunilor</b> | <b>28</b> |
| 9.1      | Structuri pentru coliziuni                   | 28        |
| 9.1.1    | Camera collider                              | 28        |
| 9.1.2    | Penguin collider                             | 28        |
| 9.2      | Inițializarea collider-elor                  | 29        |
| 9.3      | Detectarea coliziunilor                      | 29        |
| 9.4      | Rezolvarea coliziunilor                      | 29        |
| 9.5      | Integrare în loop-ul principal               | 30        |
| 9.6      | Parametri Sistem Coliziuni                   | 31        |
| 9.7      | Avantaje Sphere Colliders                    | 31        |

|           |                                                                |           |
|-----------|----------------------------------------------------------------|-----------|
| <b>10</b> | <b>Sistemul de particule pentru implementarea focului</b>      | <b>31</b> |
| 10.1      | Structura particulei . . . . .                                 | 31        |
| 10.2      | Inițializarea buffer-elor OpenGL . . . . .                     | 32        |
| 10.3      | Generarea particulelor . . . . .                               | 33        |
| 10.3.1    | Funcția respawnParticle . . . . .                              | 33        |
| 10.4      | Actualizare particule . . . . .                                | 34        |
| 10.5      | Vertex Shader - Billboard Rendering . . . . .                  | 36        |
| 10.6      | Fragment Shader - Gradient circular . . . . .                  | 36        |
| 10.7      | Depth Sorting pentru transparență . . . . .                    | 37        |
| 10.8      | Randare particule . . . . .                                    | 37        |
| 10.9      | Parametri sistem particule . . . . .                           | 39        |
| <b>11</b> | <b>Bibliografie</b>                                            | <b>39</b> |
| 11.1      | Resurse pentru vizualizarea scenei . . . . .                   | 39        |
| 11.2      | Resurse pentru Iluminare . . . . .                             | 40        |
| 11.3      | Resurse pentru Generarea Umbrelor . . . . .                    | 40        |
| 11.4      | Resurse pentru Animarea Componentelor . . . . .                | 41        |
| 11.5      | Resurse pentru Detectarea și Rezolvarea Coliziunilor . . . . . | 41        |

# 1 Introducere

Acest proiect prezintă implementarea unei scene 3D interactive fotorealiste reprezentând un campament de munte plasat pe vârful Matterhorn. Scena include multiple elemente realiste precum pinguini animați, foc cu sistem de particule, diverse obiecte de camping și un sistem avansat de iluminare și umbrire.

## 1.1 Scopul proiectului

Scopul principal al acestui proiect este demonstrarea cunoștințelor avansate de grafică pe calculator prin:

- Implementarea unui pipeline grafic complet folosind OpenGL
- Dezvoltarea unui sistem interactiv de vizualizare și control al scenei
- Aplicarea tehnicilor avansate de rendering fotorealist
- Implementarea de sisteme complexe (particule, animații, coliziuni)

## 1.2 Tehnologii utilizate

- **OpenGL** - API pentru grafica 3D
- **GLFW** - Gestionarea ferestrelor și input-ului
- **GLM** - Bibliotecă matematică pentru operații 3D
- **GLEW** - Încărcarea extensiilor OpenGL (Windows/Linux)
- **C++** - Limbajul de programare principal
- **Blender** - Gestionarea scenei, import/export obiecte

## 1.3 Structura documentației

Documentația este organizată conform cerințelor proiectului:

1. Vizualizarea scenei (scalare, translație, rotație, cameră)
2. Sistemul de iluminare (surse multiple de lumină)
3. Moduri de vizualizare (solid, wireframe, poligonal)
4. Maparea texturilor și materialele
5. Generarea umbrelor
6. Animații ale componentelor
7. Aspecte avansate (fotorealism, complexitate, algoritmi)

## 2 Vizualizarea Scenei

### 2.1 Control cu tastatura și mouse-ul

Scena poate fi explorată complet folosind tastatura și mouse-ul, oferind utilizatorului control total asupra camerei și vizualizării.

#### 2.1.1 Control cameră cu tastatura

Camera poate fi mișcată liber în toate direcțiile folosind tastele WASD:

```
1 void processMovement() {
2     if (pressedKeys[GLFW_KEY_W]) {
3         myCamera.move(gps::MOVE_FORWARD, cameraSpeed);
4     }
5     if (pressedKeys[GLFW_KEY_S]) {
6         myCamera.move(gps::MOVE_BACKWARD, cameraSpeed);
7     }
8     if (pressedKeys[GLFW_KEY_A]) {
9         myCamera.move(gps::MOVE_LEFT, cameraSpeed);
10    }
11    if (pressedKeys[GLFW_KEY_D]) {
12        myCamera.move(gps::MOVE_RIGHT, cameraSpeed);
13    }
14 }
```

Listing 1: Implementarea mișcării camerei cu tastatura

#### Comenzi disponibile:

- W - Mișcare înainte
- S - Mișcare înapoi
- A - Mișcare stânga
- D - Mișcare dreapta
- LEFT SHIFT - Sprint (viteză dublă: 30.0f vs 10.0f unități/secundă)

#### 2.1.2 Control Cameră cu Mouse-ul

Mouse-ul permite rotația camerei în stil FPS (First Person Shooter), oferind libertate completă de orientare:

```
1 void mouseCallback(GLFWwindow* window, double xpos, double ypos) {
2     if (firstMouse) {
3         lastX = xpos;
4         lastY = ypos;
5         firstMouse = false;
6     }
7
8     float xoffset = xpos - lastX;
9     float yoffset = lastY - ypos; // inversare Y
10    lastX = xpos;
11    lastY = ypos;
12
13    float sensitivity = 0.1f;
```

```

14     xoffset *= sensitivity;
15     yoffset *= sensitivity;
16
17     yaw += xoffset;
18     pitch += yoffset;
19
20     // Limitare pitch pentru a evita flip-ul camerei
21     if (pitch > 89.0f) pitch = 89.0f;
22     if (pitch < -89.0f) pitch = -89.0f;
23
24     myCamera.rotate(pitch, yaw);
25 }

```

Listing 2: Callback pentru mișcarea mouse-ului

### Caracteristici:

- Sensibilitate ajustabilă (0.1f)
- Limitare pitch la  $\pm 89^\circ$  pentru stabilitate
- Rotație continuă yaw ( $360^\circ$ )
- Cursor ascuns pentru experiență immersivă

## 2.2 Animații de prezentare

Pentru prezentarea scenei am implementat un sistem de animație cinematică bazat pe keyframe-uri, oferind o prezentare automată și fluidă a tuturor elementelor scenei.

### 2.2.1 Sistemul de Keyframe-uri

Animația folosește o serie de puncte cheie (keyframes) între care camera se interpolează smooth:

```

1 struct CameraKeyframe {
2     glm::vec3 position;    // Pozitia camerei
3     glm::vec3 target;     // Punct catre care se uita camera
4     float timestamp;      // Momentul in timp (secunde)
5 };
6
7 std::vector<CameraKeyframe> cameraPath;
8 bool isPlayingAnimation = false;

```

Listing 3: Structura keyframe-ului

### 2.2.2 Configurarea traseului camerei

Traseul animației include 12 keyframe-uri care prezintă toate zonele importante ale scenei:

```

1 void setupCameraAnimation() {
2     cameraPath.clear();
3
4     // Keyframe 1-3: Panorama initiala (5s, 10s, 15s)
5     cameraPath.push_back({
6         glm::vec3(-3173.09f, -840.953f, 6254.48f),
7         glm::vec3(-3173.28f, -840.981f, 6253.5f),

```

```

8         0.0f
9     });
10
11     // Keyframe 4-5: Apropierea de pinguini (20s, 25s)
12     cameraPath.push_back({
13         glm::vec3(-2610.85f, -874.628f, 5707.93f),
14         glm::vec3(-2610.39f, -874.686f, 5707.05f),
15         20.0f
16     });
17
18     // Keyframe 6-11: Explorarea zonelor cu obiecte
19     // (cortul, focul, pinguinii din zona indepartata)
20     // ... [vezi cod complet pentru toate keyframe-urile]
21 }

```

Listing 4: Configurarea animației de prezentare

### 2.2.3 Interpolarea fluidă

**Interpolarea liniară** În implementarea actuală, camera se mișcă fluid între keyframe-uri folosind interpolarea liniară (lerp):

```

1 void updateCameraAnimation(float currentTime) {
2     if (!isPlayingAnimation || cameraPath.size() < 2) return;
3
4     float animTime = currentTime - animationStartTime;
5
6     // Loop animatiei
7     float totalDuration = cameraPath.back().timestamp;
8     if (animTime > totalDuration) {
9         animationStartTime = currentTime;
10        animTime = 0.0f;
11    }
12
13    // Gaseste keyframe-urile curente
14    int nextKeyframe = 0;
15    for (size_t i = 0; i < cameraPath.size(); i++) {
16        if (cameraPath[i].timestamp > animTime) {
17            nextKeyframe = i;
18            break;
19        }
20    }
21
22    int prevKeyframe = nextKeyframe - 1;
23
24    // Calculeaza factorul de interpolare
25    float t1 = cameraPath[prevKeyframe].timestamp;
26    float t2 = cameraPath[nextKeyframe].timestamp;
27    float t = (animTime - t1) / (t2 - t1);
28
29    // Interpolare liniara (LERP)
30    glm::vec3 pos = glm::mix(
31        cameraPath[prevKeyframe].position,
32        cameraPath[nextKeyframe].position,
33        t
34    );
35
36    glm::vec3 target = glm::mix(

```



```

37     cameraPath[prevKeyframe].target,
38     cameraPath[nextKeyframe].target,
39     t
40 );
41
42 // Update camera
43 myCamera.setCameraPosition(pos);
44
45 // Calculeaza directia
46 glm::vec3 direction = glm::normalize(target - pos);
47 pitch = glm::degrees(asin(direction.y));
48 yaw = glm::degrees(atan2(direction.z, direction.x));
49
50 myCamera.rotate(pitch, yaw);
51 }

```

Listing 5: Funcția de update a animației cu interpolare liniară

### Formula de interpolare liniară:

$$P(t) = P_1 \cdot (1 - t) + P_2 \cdot t, \quad t \in [0, 1] \quad (1)$$

unde  $P_1$  și  $P_2$  sunt keyframe-urile consecutive, iar  $t$  reprezintă progresul între ele.

#### 2.2.4 Controlul animației

- K - Start/Stop animație
- P - Printează poziția curentă (pentru adăugarea de keyframe-uri noi)

```

1 if (key == GLFW_KEY_K && action == GLFW_PRESS) {
2     isPlayingAnimation = !isPlayingAnimation;
3     if (isPlayingAnimation) {
4         animationStartTime = glfwGetTime();
5         std::cout << "Animation STARTED" << std::endl;
6     } else {
7         std::cout << "Animation STOPPED" << std::endl;
8     }
9 }

```

Listing 6: Toggle animație în keyboardCallback

## 2.3 Matricele de transformare

Proiectul folosește cele trei matrice fundamentale pentru transformările 3D:

### 2.3.1 Matricea model

Transformă obiectele din spațiul local în spațiul lumii (world space):

```

1 model = glm::mat4(1.0f); // Matricea identitate
2 model = glm::rotate(model, glm::radians(angle),
3                     glm::vec3(0.0f, 1.0f, 0.0f));
4 modelLoc = glGetUniformLocation(shader.shaderProgram, "model");
5 glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));

```

Listing 7: Inițializarea matricei model

### 2.3.2 Matricea view

Transformă din spațiul lumii în spațiul camerei:

```
1 view = myCamera.getViewMatrix();
2 viewLoc = glGetUniformLocation(shader.shaderProgram, "view");
3 glUniformMatrix4fv(viewLoc, 1, GL_FALSE, glm::value_ptr(view));
```

Listing 8: Actualizarea matricei view

### 2.3.3 Matricea projection

Transformă din spațiul camerei în clip space (perspectivă):

```
1 projection = glm::perspective(
2     glm::radians(55.0f), // FOV (Field of View)
3     (float)retina_width / (float)retina_height, // Aspect ratio
4     0.1f, // Near plane
5     1000000.0f // Far plane (pentru munti mari)
6 );
7 projLoc = glGetUniformLocation(shader.shaderProgram, "projection");
8 glUniformMatrix4fv(projLoc, 1, GL_FALSE, glm::value_ptr(projection));
```

Listing 9: Inițializarea matricei projection

## 2.4 Pipeline-ul de transformare complet

Local Space (Vertex)  
    ↓ [Model Matrix]  
World Space  
    ↓ [View Matrix]  
View/Camera Space  
    ↓ [Projection Matrix]  
Clip Space  
    ↓ [Perspective Division]  
NDC (Normalized Device Coordinates)  
    ↓ [Viewport Transform]  
Screen Space (Pixel)

Figura 1: Pipeline-ul de transformare OpenGL

## 2.5 Rezumat funcționalități vizualizare

| Funcționalitate     | Implementare                         |
|---------------------|--------------------------------------|
| Mișcare cameră      | WASD + Mouse                         |
| Sprint              | LEFT SHIFT (viteză x3)               |
| Rotație scenă       | Q/E ( $\pm 1^\circ$ per frame)       |
| Animatie prezentare | K (toggle), 12 keyframes, 65s durată |
| Interpolare         | Linear (glm::mix)                    |
| FOV                 | $55^\circ$                           |
| Far plane           | 1,000,000 unități (pentru munte)     |

Tabela 1: Rezumat comenzi și parametri vizualizare

## 3 Sursele de lumină

Scena utilizează un sistem complex de iluminare cu două surse diferite de lumină: o lumină direcțională (soarele) și o lumină punctiformă (focul de tabără). Ambele sunt implementate în fragment shader și contribuie la realismul scenei.

### 3.1 Lumina direcțională (Soarele)

Lumina direcțională simulează soarele și afectează uniform întreaga scenă, având o direcție constantă dar fără o poziție specifică.

#### 3.1.1 Inițializare și configurare

```
1 // In initUniforms()
2 lightDir = glm::normalize(glm::vec3(0.0f, -1.0f, -0.3f));
3 lightColor = glm::vec3(1.0f, 1.0f, 0.95f); // lumina alba-galbui
4
5 lightDirLoc = glGetUniformLocation(lightShader.shaderProgram,
6                                     "lightDirEye");
7 lightColorLoc = glGetUniformLocation(lightShader.shaderProgram,
8                                     "lightColor");
9
10 glUniform3fv(lightDirLoc, 1, glm::value_ptr(lightDir));
11 glUniform3fv(lightColorLoc, 1, glm::value_ptr(lightColor));
```

Listing 10: Inițializarea luminii direcționale

#### 3.1.2 Control interactiv al direcției

Utilizatorul poate ajusta direcția soarelui folosind săgețile:

```
1 if (glfwGetKey(window, GLFW_KEY_UP) == GLFW_PRESS)
2     lightDir.y += 0.01f;
3 if (glfwGetKey(window, GLFW_KEY_DOWN) == GLFW_PRESS)
4     lightDir.y -= 0.01f;
5 if (glfwGetKey(window, GLFW_KEY_LEFT) == GLFW_PRESS)
6     lightDir.x -= 0.01f;
7 if (glfwGetKey(window, GLFW_KEY_RIGHT) == GLFW_PRESS)
```

```

8     lightDir.x += 0.01f;
9
10    // Normalizare pentru a mentine directia corecta
11    lightDir = glm::normalize(lightDir);

```

Listing 11: Controlul direcției luminii în keyboardCallback

### 3.1.3 Ciclul zi/noapte

Implementarea permite comutarea între zi și noapte, schimbând culoarea luminii solare:

```

1 // Culori definite
2 glm::vec3 daySunColor = glm::vec3(1.0f, 1.0f, 0.95f); // alb-galben
3 glm::vec3 nightSunColor = glm::vec3(0.17f); // gri inchis
4
5 // Control cu tastele N si M
6 if (key == GLFW_KEY_N && action == GLFW_PRESS) {
7     sunOn = false; // Noapte
8     loadSky(); // Schimba si sky dome-ul
9 }
10 if (key == GLFW_KEY_M && action == GLFW_PRESS) {
11     sunOn = true; // Zi
12     loadSky();
13 }
14
15 // In renderScene() - actualizare culoare
16 glm::vec3 currentLightColor = sunOn ? daySunColor : nightSunColor;
17 glUniform3fv(lightColorLoc, 1, glm::value_ptr(currentLightColor));

```

Listing 12: Ciclul zi/noapte

### 3.1.4 Implementare în Shader

În vertex shader, normala și poziția sunt transformate în view space:

```

1 // Transformare normala in view space
2 normalEye = mat3(transpose(inverse(view * model))) * vertexNormal;
3
4 // Transformare pozitie in view space
5 vec4 worldPos = model * vec4(vertexPosition, 1.0);
6 vec4 posEye = view * worldPos;
7 fragPosEye = posEye.xyz;

```

Listing 13: Fragment din lightShader.vert

În fragment shader, se calculează iluminarea Phong:

```

1 // Componentele luminii
2 vec3 ambient = 0.2 * lightColor;
3
4 vec3 N = normalize(normalEye);
5 vec3 L = normalize(-lightDirEye); // inversare directie
6 vec3 V = normalize(-fragPosEye); // catre camera
7 vec3 R = reflect(-L, N);
8
9 // Diffuse (Lambert)
10 float diff = max(dot(N, L), 0.0);
11 vec3 diffuse = diff * lightColor;

```

```

12
13 // Specular (Phong)
14 float spec = pow(max(dot(V, R), 0.0), shininess);
15 vec3 specular = specularStrength * spec * lightColor;
16
17 // Aplicare pe textura
18 vec3 texColor = texture(diffuseTexture, passTexture).rgb;
19 vec3 result = ambient * texColor;
20 result += (1.0 - shadow) * (diffuse + specular) * texColor;

```

Listing 14: Iluminare direcțională în lightShader.frag

### Modelul Phong:

$$I = I_{ambient} + I_{diffuse} + I_{specular} \quad (2)$$

unde:

- $I_{ambient} = 0.2 \cdot lightColor$  (lumină ambientă constantă)
- $I_{diffuse} = \max(\vec{N} \cdot \vec{L}, 0) \cdot lightColor$  (Lambert)
- $I_{specular} = (\max(\vec{V} \cdot \vec{R}, 0))^{shininess} \cdot specularStrength \cdot lightColor$

## 3.2 Lumina punctiformă (focul)

Lumina punctiformă simulează focul de tabără și are intensitate care scade cu distanța (attenuation).

### 3.2.1 Poziționare și Configurare

```

1 // Pozitia focului in lume
2 glm::vec3 firePos = glm::vec3(-2096.814209f, -980.905457f, 5921.6f);
3
4 // Culori diferite zi/noapte
5 glm::vec3 dayFireColor = glm::vec3(0.9f, 0.5f, 0.2f); // portocaliu
6 glm::vec3 nightFireColor = glm::vec3(3.0f, 2.0f, 0.8f); // mai intens
7
8 // In initUniforms()
9 pointLightColor = glm::vec3(3.0f, 2.0f, 0.8f);
10 lightPosLoc = glGetUniformLocation(lightShader.shaderProgram,
11                                     "lightPosEye");
12 pointLightColorLoc = glGetUniformLocation(lightShader.shaderProgram,
13                                             "pointLightColor");
14
15 // Transformare pozitie in view space
16 glm::vec4 lightPosEye = view * glm::vec4(firePos, 1.0f);
17 glUniform3fv(lightPosLoc, 1, glm::value_ptr(glm::vec3(lightPosEye)));
18 glUniform3fv(pointLightColorLoc, 1, glm::value_ptr(pointLightColor));

```

Listing 15: Inițializarea luminii punctiforme

### 3.2.2 Atenuare (Attenuation)

Intensitatea luminii scade cu distanța folosind formula de atenuare:

```

1 // Constante de atenuare
2 float constant = 1.0f;
3 float linear = 0.0001f;
4 float quadratic = 0.0000025f;
5
6 // Calculare atenuare
7 vec3 lightDirN = normalize(lightPosEye - fragPosEye.xyz);
8 float dist = length(lightPosEye - fragPosEye.xyz);
9 float att = 1.0f / (constant + linear * dist + quadratic * (dist * dist)
   );

```

Listing 16: Atenuare în lightShader.frag

**Formula de atenuare:**

$$F_{att} = \frac{1}{K_c + K_l \cdot d + K_q \cdot d^2} \quad (3)$$

unde:

- $K_c = 1.0$  (constant)
- $K_l = 0.0001$  (linear)
- $K_q = 0.0000025$  (quadratic)
- $d$  = distanța de la lumină la fragment

### 3.2.3 Efect de Pâlpâire (Flickering)

Pentru un efect realist de foc, intensitatea luminii variază în timp:

```

1 // Componente sinusoidale cu frecvente diferite
2 float flicker1 = sin(time * 2.5 + 0.3) * 0.5 + 0.5;
3 float flicker2 = sin(time * 6.7 + 1.2) * 0.3 + 0.7;
4 float flicker3 = sin(time * 11.3 + 2.1) * 0.2 + 0.8;
5 float flicker4 = sin(time * 17.8 + 0.7) * 0.1 + 0.9;
6
7 // Intensitate combinata (65% - 100%)
8 float flickerIntensity = flicker1 * flicker2 * flicker3 * flicker4;
9 flickerIntensity = 0.65 + flickerIntensity * 0.35;
10
11 // Variatie de culoare (portocaliu-galben-rosu)
12 float colorShift = sin(time * 1.8) * 0.15 + 0.85;
13 vec3 fireColorVariation = vec3(
14     1.0, // rosu constant
15     0.5 + colorShift * 0.3, // verde (portocaliu-galben)
16     0.2 * colorShift // albastru minimal
17 );
18
19 // Aplicare efecte
20 vec3 pointLightColor_dynamic = pointLightColor * flickerIntensity
21     * fireColorVariation;

```

Listing 17: Flickering dinamic al focului

**Explicație tehnică:**

- 4 unde sinusoidale cu frecvențe diferite (2.5, 6.7, 11.3, 17.8 Hz)

- Offset-uri de fază (0.3, 1.2, 2.1, 0.7) pentru variație naturală
- Amplitudini descrescătoare (50%, 30%, 20%, 10%) pentru realism
- Shift de culoare separat pentru tranzițiile portocaliu-galben-roșu

### 3.2.4 Iluminare Phong pentru lumina punctiformă

```

1 // Diffuse
2 float diff_point = max(dot(N, lightDirN), 0.0);
3 vec3 diffuse_point = diff_point * pointLightColor_dynamic;
4
5 // Specular
6 vec3 R_point = reflect(-lightDirN, N);
7 float spec_point = pow(max(dot(V, R_point), 0.0), shininess);
8 vec3 specular_point = specularStrength * spec_point
9                       * pointLightColor_dynamic;
10
11 // Ambient
12 vec3 ambient_point = 0.2 * pointLightColor_dynamic;
13
14 // Aplicare atenuare
15 diffuse_point *= att;
16 specular_point *= att;
17 ambient_point *= att;

```

Listing 18: Calculul iluminării punctiforme

## 3.3 Combinarea surselor de lumină

Cele două surse sunt combinate în rezultatul final:

```

1 vec3 result = ambient * textColor + ambient_point * textColor;
2 result += (1.0 - shadow) * (diffuse + specular) * textColor;
3 result += (diffuse_point + specular_point) * textColor;
4 result *= objectLightMultiplier; // Ajustare per-obiect
5
6 fragmentColour = vec4(result, 1.0);

```

Listing 19: Combinare finale în fragment shader

## 3.4 Parametri materiale per obiect

Fiecare obiect poate avea proprietăți diferite:

```

1 // Pinguini - material mat
2 glUniform1f(shininessLoc, 8.0f);
3 glUniform1f(specStrengthLoc, 0.15f);
4
5 // Skis - material lucios
6 glUniform1f(shininessLoc, 64.0f);
7 glUniform1f(specStrengthLoc, 0.6f);
8
9 // Matterhorn - material standard
10 glUniform1f(shininessLoc, 32.0f);
11 glUniform1f(specStrengthLoc, 0.3f);

```

### 3.5 Rezumat Sistem Iluminare

| Caracteristică  | Lumină Direcțională     | Lumină Punctiformă                     |
|-----------------|-------------------------|----------------------------------------|
| Tip             | Directional (soare)     | Point light (foc)                      |
| Poziție         | Infinit (doar direcție) | firePos (world space)                  |
| Culoare zi      | (1.0, 1.0, 0.95)        | (0.9, 0.5, 0.2)                        |
| Culoare noapte  | (0.17, 0.17, 0.17)      | (3.0, 2.0, 0.8)                        |
| Atenuare        | Nu                      | Da ( $1/(c+l \cdot d + q \cdot d^2)$ ) |
| Efecte dinamice | Nu                      | Flickering + color shift               |
| Control         | Arrow keys + N/M        | Automat (bazat pe timp)                |
| Umbre           | Da (shadow mapping)     | Nu                                     |

Tabela 2: Comparație surse de lumină

## 4 Moduri de Vizualizare

Scena poate fi vizualizată în trei moduri diferite folosind funcția `glPolygonMode()`.

### 4.1 Implementare

```

1 if (pressedKeys[GLFW_KEY_1]) {
2     glPolygonMode(GL_FRONT_AND_BACK, GL_FILL); // Solid
3 }
4 if (pressedKeys[GLFW_KEY_2]) {
5     glPolygonMode(GL_FRONT_AND_BACK, GL_LINE); // Wireframe
6 }
7 if (pressedKeys[GLFW_KEY_3]) {
8     glPolygonMode(GL_FRONT_AND_BACK, GL_POINT); // Poligonal
9 }

```

Listing 21: Control moduri de vizualizare

### 4.2 Descriere Moduri

- **Tastă 1 - Modul Solid (`GL_FILL`):** Modul implicit. Poligoanele sunt umplute complet, afișând texturi, iluminare și umbre.
- **Tastă 2 - Modul Wireframe (`GL_LINE`):** Doar marginile poligoanelor sunt desenate. Util pentru analiza topologiei mesh-ului și debugging.
- **Tastă 3 - Modul Poligonal (`GL_POINT`):** Doar vertex-urile sunt afișate ca puncte. Util pentru analiza distribuției geometrice.



## 5 Maparea Texturilor și Materiale

### 5.1 Încărcarea Texturilor

Texturile sunt încărcate folosind metoda `ReadTextureFromFile()` din clasa `Model3D`:

```
1 // Matterhorn
2 matterhornTexture = matterhorn.ReadTextureFromFile(
3     "models/Matterhorn/Matterhorn.jpg");
4
5 // Sky dome
6 skyTexture = sky.ReadTextureFromFile(
7     "models/SkyDome/skydomeBIG.png");
8
9 // Pinguini
10 penguinTexture = penguinBody.ReadTextureFromFile(
11     "models/penguin/Penguin Diffuse Color.png");
12
13 // Obiecte
14 tentTexture = tent.ReadTextureFromFile(
15     "models/Tent/tentTexture.jpg");
16 skisTexture = skis.ReadTextureFromFile(
17     "models/skis/skisTexture.jpg");
```

Listing 22: Încărcare texturi în `initModels()`

### 5.2 Aplicarea Texturilor

Texturile sunt aplicate prin binding la texture unit 0 înainte de draw:

```
1 // Activare texture unit
2 glActiveTexture(GL_TEXTURE0);
3 glUniform1i(glGetUniformLocation(shader.shaderProgram,
4     "diffuseTexture"), 0);
5
6 // Bind textura
7 glBindTexture(GL_TEXTURE_2D, matterhornTexture);
8
9 // Draw model
10 matterhorn.Draw(shader);
```

Listing 23: Aplicare texturi în shader

În fragment shader, textura este eșantionată folosind coordonatele UV:

```
1 uniform sampler2D diffuseTexture;
2 in vec2 passTexture;
3
4 void main() {
5     vec3 textColor = texture(diffuseTexture, passTexture).rgb;
6     // ... aplicare iluminare pe textColor ...
7 }
```

Listing 24: Sampling texturi în `lightShader.frag`

## 5.3 Proprietăți Materiale

Fiecare obiect are parametri materiali specifici (shininess și specular strength):

```
1 // Pinguini - material mat
2 glUniform1f(shininessLoc, 8.0f);
3 glUniform1f(specStrengthLoc, 0.15f);
4
5 // Skis - material lucios
6 glUniform1f(shininessLoc, 64.0f);
7 glUniform1f(specStrengthLoc, 0.6f);
8
9 // Matterhorn - material standard
10 glUniform1f(shininessLoc, 32.0f);
11 glUniform1f(specStrengthLoc, 0.3f);
```

Listing 25: Configurare materiale per obiect

| Obiect     | Shininess | Specular | Aspect             |
|------------|-----------|----------|--------------------|
| Pinguini   | 8.0       | 0.15     | Mat, non-reflectiv |
| Matterhorn | 32.0      | 0.3      | Standard, rocă     |
| Skis       | 64.0      | 0.6      | Lucios, reflectiv  |
| Goggles    | 64.0      | 0.3      | Lucios moderat     |

Tabela 3: Parametri materiale

## 5.4 Coordonate UV

Coordonatele de textură sunt preluate din fișierele OBJ și transmise prin vertex shader:

```
1 // Vertex shader
2 layout(location=2) in vec2 vertexTexture;
3 out vec2 passTexture;
4
5 void main() {
6     passTexture = vertexTexture;
7     // ...
8 }
9
10 // Fragment shader
11 in vec2 passTexture;
12 vec3 color = texture(diffuseTexture, passTexture).rgb;
```

Listing 26: Transmite coordonate UV

# 6 Generarea Umbrelor

Proiectul implementează umbre folosind tehnica **Shadow Mapping** pentru lumina direcțională (soarele).

## 6.1 Shadow Map - Configurare

Shadow map-ul este o textură de adâncime randată din perspectiva luminii:

```

1  const unsigned int SHADOW_WIDTH = 8192, SHADOW_HEIGHT = 8192;
2  GLuint shadowMapFBO;
3  GLuint shadowMapTexture;
4
5  void initShadowMap() {
6      // Creare framebuffer
7      glGenFramebuffers(1, &shadowMapFBO);
8
9      // Creare depth texture
10     glGenTextures(1, &shadowMapTexture);
11     glBindTexture(GL_TEXTURE_2D, shadowMapTexture);
12     glTexImage2D(GL_TEXTURE_2D, 0, GL_DEPTH_COMPONENT,
13                 SHADOW_WIDTH, SHADOW_HEIGHT, 0,
14                 GL_DEPTH_COMPONENT, GL_FLOAT, NULL);
15
16     // Parametri textura
17     glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_NEAREST);
18     glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_NEAREST);
19     glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_CLAMP_TO_BORDER
20 );
21     glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_CLAMP_TO_BORDER
22 );
23
24     float borderColor[] = {1.0f, 1.0f, 1.0f, 1.0f};
25     glTexParameterfv(GL_TEXTURE_2D, GL_TEXTURE_BORDER_COLOR, borderColor
26 );
27
28     // Atasare depth texture la FBO
29     glBindFramebuffer(GL_FRAMEBUFFER, shadowMapFBO);
30     glFramebufferTexture2D(GL_FRAMEBUFFER, GL_DEPTH_ATTACHMENT,
31                           GL_TEXTURE_2D, shadowMapTexture, 0);
32     glDrawBuffer(GL_NONE);
33     glReadBuffer(GL_NONE);
34     glBindFramebuffer(GL_FRAMEBUFFER, 0);
35 }

```

Listing 27: Inițializare shadow map

## 6.2 Light Space Matrix

Matricea care transformă din world space în light space (perspectiva luminii):

```

1  void calculateLightSpaceMatrix() {
2      // Pozitia luminii (departe in directia luminii)
3      glm::vec3 lightPos = -lightDir * 15000.0f;
4
5      // Proiectie ortografica (lumina directionala)
6      glm::mat4 lightProjection = glm::ortho(
7          -20000.0f, 20000.0f, // left, right
8          -20000.0f, 20000.0f, // bottom, top
9          1.0f, 50000.0f       // near, far
10 );
11
12     // View matrix din perspectiva luminii
13     glm::mat4 lightView = glm::lookAt(
14         lightPos, // pozitie lumina
15         glm::vec3(0.0f), // priveste spre origine

```

```

16         glm::vec3(0.0f, 1.0f, 0.0f) // up vector
17     );
18
19     lightSpaceMatrix = lightProjection * lightView;
20 }

```

Listing 28: Calculul light space matrix

## 6.3 Shadow Pass - Depth Map Rendering

Primul pass: randarea scenei din perspectiva luminii pentru a genera depth map-ul.

### 6.3.1 Depth Map Vertex Shader

```

1 #version 410 core
2 layout(location = 0) in vec3 vertexPosition;
3
4 uniform mat4 model;
5 uniform mat4 lightSpaceMatrix;
6
7 void main()
8 {
9     gl_Position = lightSpaceMatrix * model * vec4(vertexPosition, 1.0);
10 }

```

Listing 29: depthMap.vert

### 6.3.2 Depth Map Fragment Shader

```

1 #version 410 core
2 void main()
3 {
4     // OpenGL scrie depth automat in gl_FragDepth
5 }

```

Listing 30: depthMap.frag

### 6.3.3 Randare Depth Map

```

1 if (renderShadows) {
2     // ===== SHADOW PASS =====
3     calculateLightSpaceMatrix();
4
5     glViewport(0, 0, SHADOW_WIDTH, SHADOW_HEIGHT);
6     glBindFramebuffer(GL_FRAMEBUFFER, shadowMapFBO);
7     glClear(GL_DEPTH_BUFFER_BIT);
8
9     // Render scene din perspectiva luminii
10    depthMapShader.useShaderProgram();
11    renderDepthMap(depthMapShader, false);
12
13    glBindFramebuffer(GL_FRAMEBUFFER, 0);
14    glViewport(0, 0, retina_width, retina_height);
15 }

```

## 6.4 Scene Rendering Pass

Al doilea pass: randarea scenei normale cu aplicarea umbrelor.

### 6.4.1 Light Shader - Vertex

Calculează poziția fragmentului în light space:

```
1 out vec4 fragPosLightSpace;
2 uniform mat4 lightSpaceMatrix;
3
4 void main()
5 {
6     vec4 worldPos = model * vec4(vertexPosition, 1.0);
7
8     // Pozitia in light space pentru shadow mapping
9     fragPosLightSpace = lightSpaceMatrix * worldPos;
10
11     // ... rest of shader
12 }
```

Listing 32: lightShader.vert - fragment relevant

### 6.4.2 Shadow Calculation în Fragment Shader

Funcția care determină dacă un fragment este în umbră:

```
1 uniform sampler2D shadowMap;
2
3 float ShadowCalculation(vec4 fragPosLightSpace, vec3 normal, vec3
4     lightDir)
5 {
6     // Perspective divide
7     vec3 projCoords = fragPosLightSpace.xyz / fragPosLightSpace.w;
8
9     // Transformare in range [0,1]
10    projCoords = projCoords * 0.5 + 0.5;
11
12    // In afara light frustum      fara umbra
13    if(projCoords.z > 1.0)
14        return 0.0;
15
16    // Adancimea curenta a fragmentului
17    float currentDepth = projCoords.z;
18
19    // Bias pentru a preveni shadow acne
20    float bias = max(0.002 * (1.0 - dot(normal, lightDir)), 0.0005);
21
22    // PCF (Percentage Closer Filtering) pentru umbre soft
23    float shadow = 0.0;
24    vec2 texelSize = 1.0 / textureSize(shadowMap, 0);
25
26    // Kernel 5x5
```

```

26   for(int x = -2; x <= 2; ++x)
27   {
28       for(int y = -2; y <= 2; ++y)
29       {
30           float pcfDepth = texture(shadowMap,
31                                   projCoords.xy + vec2(x, y) * texelSize).r;
32           shadow += currentDepth - bias > pcfDepth ? 1.0 : 0.0;
33       }
34   }
35
36   shadow /= 25.0; // Media peste 25 samples
37
38   return shadow;
39 }

```

Listing 33: Shadow calculation în lightShader.frag

### 6.4.3 Aplicarea Umbrelor

```

1 void main()
2 {
3     // ... calcul iluminare ...
4
5     // Calculeaza shadow factor
6     float shadow = ShadowCalculation(fragPosLightSpace, N, L);
7
8     // Aplica umbra doar pe lumina directionala
9     vec3 result = ambient * textColor + ambient_point * textColor;
10    result += (1.0 - shadow) * (diffuse + specular) * textColor;
11    result += (diffuse_point + specular_point) * textColor;
12
13    fragmentColour = vec4(result, 1.0);
14 }

```

Listing 34: Utilizare shadow în main()

## 6.5 Tehnici Anti-Aliasing pentru Umbre

### 6.5.1 Shadow Acne Prevention

Bias-ul previne artefactele de "shadow acne" (pattern în umbră):

```

1 float bias = max(0.002 * (1.0 - dot(normal, lightDir)), 0.0005);

```

Listing 35: Bias adaptiv

**Formula:** Bias-ul crește când suprafața este mai oblică față de lumină.

### 6.5.2 PCF (Percentage Closer Filtering)

PCF estompează marginile umbrelor prin eșantionare multiplă:

- Kernel  $5 \times 5 = 25$  samples
- Fiecare sample verifică depth

- Media rezultatelor → umbră soft

**Rezultat:** Margini de umbră naturale, nu hard edges.

## 6.6 Control Umbre

```
1 if (key == GLFW_KEY_O && action == GLFW_PRESS) {
2     renderShadows = !renderShadows;
3 }
```

Listing 36: Toggle shadows cu tasta O

## 6.7 Rezumat Tehnic

| Parametru            | Valoare                     |
|----------------------|-----------------------------|
| Rezoluție shadow map | 8192×8192                   |
| Tip proiecție        | Ortografică                 |
| PCF kernel           | 5×5 (25 samples)            |
| Bias minim           | 0.0005                      |
| Bias maxim           | 0.002                       |
| Control              | Tastă O (ON/OFF)            |
| Aplică pe            | Lumină direcțională (soare) |

Tabela 4: Parametri shadow mapping

# 7 Animarea Componentelor

Proiectul implementează animații ale componentelor obiectelor, exemplificat prin pinguinul animat cu aripi mobile.

## 7.1 Structura modelului animat

Pinguinul este împărțit în trei componente separate pentru animație:

```
1 gps::Model3D penguinBody;
2 gps::Model3D penguinWingL;
3 gps::Model3D penguinWingR;
4
5 // In initModels()
6 penguinBody.LoadModel("models/penguin/penguinBody.obj");
7 penguinWingL.LoadModel("models/penguin/penguinWingL.obj");
8 penguinWingR.LoadModel("models/penguin/penguinWingR.obj");
9
10 penguinTexture = penguinBody.ReadTextureFromFile(
11     "models/penguin/Penguin Diffuse Color.png");
```

Listing 37: Încărcarea componentelor pinguinului

## 7.2 Puncte pivot pentru rotație

Fiecare aripă se rotește în jurul unui punct pivot specific (articulația aripii):

```
1 // Pivot aripa stanga (pozitie in world space)
2 glm::vec3 wingLPivot = glm::vec3(-2068.25f, -884.082f, 5489.76f);
3
4 // Pivot aripa dreapta
5 glm::vec3 wingRPivot = glm::vec3(-2008.24f, -877.652f, 5558.12f);
```

Listing 38: Definirea punctelor pivot

## 7.3 Calculul animației cu funcții sinusoidale

Unghiul de rotație al aripilor variază sinusoidal în timp:

```
1 float t = glfwGetTime(); // Timpul curent in secunde
2
3 // Func ie sinusoidal pentru mi care natural
4 // Frecven : 4.0 Hz (4 b t i/secund )
5 // Amplitudine: 30 de grade
6 float wingAngle = sin(t * 4.0f) * glm::radians(30.0f);
```

Listing 39: Calculul unghiului de rotație

**Formula matematică:**

$$\theta(t) = A \cdot \sin(\omega \cdot t) \quad (4)$$

unde:

- $\theta(t)$  = unghiul la momentul  $t$
- $A = 30$  = amplitudinea (rang de mișcare)
- $\omega = 4.0$  Hz = frecvența (viteza animației)
- $t$  = timpul în secunde

## 7.4 Transformări Ierarhice pentru Aripă

Transformările sunt aplicate în ordinea: **translație** → **rotație** → **translație inversă**.

### 7.4.1 Aripa stângă

```
1 glm::mat4 wingLModel = bodyModel *
2   glm::translate(glm::mat4(1.0f), wingLPivot) *           // (1)
3   glm::rotate(glm::mat4(1.0f), wingAngle,                 // (2)
4     glm::vec3(0, 0, 1)) *
5   glm::translate(glm::mat4(1.0f), -wingLPivot);           // (3)
6
7 glUniformMatrix4fv(modelLocL, 1, GL_FALSE, glm::value_ptr(wingLModel));
8 normalMatrix = glm::mat3(glm::transpose(glm::inverse(view * wingLModel))
9   );
10 glUniformMatrix3fv(normalMatrixLocL, 1, GL_FALSE, glm::value_ptr(
   normalMatrix));
10 penguinWingL.Draw(shader);
```

Listing 40: Transformarea aripii stângi



### Pași transformare:

1. `translate(wingLPivot)`: Mută pivotul în origine
2. `rotate(wingAngle, Z)`: Rotește în jurul axei Z
3. `translate(-wingLPivot)`: Readuce în poziția originală

#### 7.4.2 Aripa dreaptă

```
1 glm::mat4 wingRModel = bodyModel *
2   glm::translate(glm::mat4(1.0f), wingRPivot) *
3   glm::rotate(glm::mat4(1.0f), -wingAngle,           // unghi negativ
4     glm::vec3(0, 0, 1)) *
5   glm::translate(glm::mat4(1.0f), -wingRPivot);
6
7 glUniformMatrix4fv(modelLocL, 1, GL_FALSE, glm::value_ptr(wingRModel));
8 normalMatrix = glm::mat3(glm::transpose(glm::inverse(view * wingRModel))
9   );
10 glUniformMatrix3fv(normalMatrixLocL, 1, GL_FALSE, glm::value_ptr(
   normalMatrix));
10 penguinWingR.Draw(shader);
```

Listing 41: Transformarea aripii drepte

**Notă:** Aripa dreaptă folosește `-wingAngle` pentru mișcare simetrică (când stânga merge sus, dreapta merge jos).

## 7.5 Randarea corpului

Corpul rămâne static și servește ca referință pentru aripi:

```
1 glm::vec3 penguinPos = glm::vec3(0.0f, 0.0f, 0.0f);
2 glm::mat4 bodyModel = glm::translate(glm::mat4(1.0f), penguinPos);
3
4 glUniformMatrix4fv(modelLocL, 1, GL_FALSE, glm::value_ptr(bodyModel));
5 normalMatrix = glm::mat3(glm::transpose(glm::inverse(view * bodyModel)))
6   ;
7 glUniformMatrix3fv(normalMatrixLocL, 1, GL_FALSE, glm::value_ptr(
   normalMatrix));
7 penguinBody.Draw(shader);
```

Listing 42: Randarea corpului pinguinului

## 7.6 Animație în shadow pass

Același algoritm de animație este aplicat și în shadow pass pentru consistență:

```
1 float t = glfwGetTime();
2 float wingAngle = sin(t * 4.0f) * glm::radians(30.0f);
3
4 // Aripa stanga - shadow pass
5 glm::mat4 wingLModel = objectModel *
6   glm::translate(glm::mat4(1.0f), wingLPivot) *
7   glm::rotate(glm::mat4(1.0f), wingAngle, glm::vec3(0, 0, 1)) *
8   glm::translate(glm::mat4(1.0f), -wingLPivot);
```

```

9
10 glm::UniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(wingLModel));
11 penguinWingL.Draw(shader);
12
13 // Acela i proces pentru aripa dreapta

```

Listing 43: Animație în renderDepthMap()

## 7.7 Parametri animație

| Parametru    | Valoare        | Descriere                   |
|--------------|----------------|-----------------------------|
| Frecvență    | 4.0 Hz         | 4 bătăi de aripi pe secundă |
| Amplitudine  | $\pm 30^\circ$ | Range-ul de rotație         |
| Axa rotație  | Z              | Vertical (sus-jos)          |
| Sincronizare | Opusă          | Aripi în contra-fază        |

Tabela 5: Parametri animație aripi penguin

## 7.8 Diagrama Transformărilor

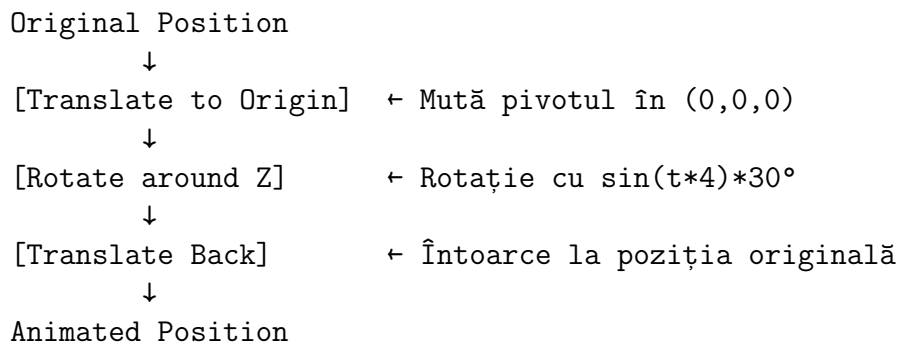


Figura 2: Pipeline transformări pentru animația aripilor

# 8 Generarea Dinamică de Obiecte

Proiectul implementează generarea dinamică a obiectelor, exemplificat prin schimbarea cerului între zi și noapte.

## 8.1 Sistemul Zi/Noapte

Cerul (sky dome) se schimbă dinamic în funcție de starea curentă (zi sau noapte):

```

1 bool sunOn = true; // Starea curenta: true = zi, false = noapte
2
3 glm::vec3 daySunColor = glm::vec3(1.0f, 1.0f, 0.95f); // alb-galben
4 glm::vec3 nightSunColor = glm::vec3(0.17f); // gri
5     inchis
6 glm::vec3 dayFireColor = glm::vec3(0.9f, 0.5f, 0.2f); // portocaliu
7 glm::vec3 nightFireColor = glm::vec3(3.0f, 2.0f, 0.8f); // intens

```

Listing 44: Variabile pentru ciclul zi/noapte

## 8.2 Încărcarea Dinamică a Sky Dome-ului

Funcția `loadSky()` încarcă modelul și textura corespunzătoare în funcție de starea curentă:

```
1 void loadSky() {
2     if (sunOn) {
3         // Zi - cer albastru
4         sky.LoadModel("models/SkyDome/sky.obj");
5         skyTexture = sky.ReadTextureFromFile(
6             "models/SkyDome/skydomeBIG.png");
7     }
8     else {
9         // Noapte - cer cu stele
10        sky.LoadModel("models/SkyDome/nightSky.obj");
11        skyTexture = sky.ReadTextureFromFile(
12            "models/SkyDome/nightSky.jpg");
13    }
14 }
```

Listing 45: Funcția `loadSky()`

## 8.3 Control Interactiv

Utilizatorul poate comuta între zi și noapte folosind tastele N și M:

```
1 // Tasta N - Activare noapte
2 if (key == GLFW_KEY_N && action == GLFW_PRESS) {
3     sunOn = false;
4     loadSky(); // Incarcare cer nocturn
5 }
6
7 // Tasta M - Activare zi
8 if (key == GLFW_KEY_M && action == GLFW_PRESS) {
9     sunOn = true;
10    loadSky(); // Incarcare cer diurn
11 }
```

Listing 46: Control zi/noapte în `keyboardCallback`

## 8.4 Actualizarea Culorilor Luminii

Culorile surselor de lumină se schimbă automat pentru a se potrivi cu timpul zilei:

```
1 // Culoarea soarelui (lumina directionala)
2 glm::vec3 currentLightColor = sunOn ? daySunColor : nightSunColor;
3 glUniform3fv(lightColorLoc, 1, glm::value_ptr(currentLightColor));
4
5 // Culoarea focului (lumina punctiforma)
6 glm::vec3 currentFireColor = sunOn ? dayFireColor : nightFireColor;
7 glUniform3fv(pointLightColorLoc, 1, glm::value_ptr(currentFireColor));
```

Listing 47: Actualizare culori lumini în `renderScene()`

## 8.5 Efecte Vizuale Zi/Noapte

### 8.5.1 Ziua

- **Cer:** albastru cu nori (skydomeBIG.png, 8192×4096)
- **Soare:** lumină alb-gălbuie (1.0, 1.0, 0.95)
- **Foc:** portocaliu discret (0.9, 0.5, 0.2)
- **Vizibilitate:** maximă, umbre clare

### 8.5.2 Noaptea

- **Cer:** negru cu stele (nightSky.jpg)
- **Lună/Stele:** lumină gri închis (0.17, 0.17, 0.17)
- **Foc:** foarte intens (3.0, 2.0, 0.8) - sursa principală de lumină
- **Vizibilitate:** redusă, atmosferă nocturnă

## 9 Detectarea și rezolvarea coliziunilor

Proiectul implementează un sistem de detectare și rezolvare a coliziunilor între cameră și pinguini folosind **\*\*Sphere Colliders\*\***.

### 9.1 Structuri pentru coliziuni

#### 9.1.1 Camera collider

```
1 struct CameraCollider {
2     glm::vec3 position;
3     float radius;
4 };
5
6 CameraCollider cameraCollider = {
7     myCamera.getCameraPosition(),
8     2.0f // Raza de 2 unitati
9 };
```

Listing 48: Structura collider-ului camerei

#### 9.1.2 Penguin collider

```
1 struct Penguin {
2     glm::vec3 position;
3     float radius;
4     glm::vec3 velocity;
5 };
6
7 std::vector<Penguin> penguins;
```

Listing 49: Structura collider-ului pinguinului

## 9.2 Inițializarea collider-elor

Fiecare penguin din scenă primește un collider sferic:

```
1 void initPenguinsVector() {
2     penguins.clear();
3
4     float radius = 90.0f; // Raza collider penguin
5     glm::vec3 velocity = glm::vec3(0.0f);
6
7     // Penguin 1
8     penguins.push_back({
9         glm::vec3(-2050.9f, -877.6f, 5533.9f), // pozitie
10        radius,
11        velocity
12    });
13
14    // Penguin 2
15    penguins.push_back({
16        glm::vec3(7169.0f, 562.46f, 7153.5f),
17        radius,
18        velocity
19    });
20
21    // ... restul pinguinilor (total 9)
22 }
```

Listing 50: Funcția initPenguinsVector()

## 9.3 Detectarea coliziunilor

Algoritmul verifică dacă distanța dintre centre este mai mică decât suma razelor:

```
1 bool checkCameraCollision(const CameraCollider& cam,
2                           const Penguin& penguin) {
3     // Calculeaza distanta intre centre
4     float distance = glm::length(cam.position - penguin.position);
5
6     // Coliziune daca distanta < suma razelor
7     return distance < (cam.radius + penguin.radius);
8 }
```

Listing 51: Funcția checkCameraCollision()

**Formula matematică:**

$$\text{Coliziune} = \begin{cases} \text{true}, & \text{dacă } \|P_{cam} - P_{penguin}\| < r_{cam} + r_{penguin} \\ \text{false}, & \text{altfel} \end{cases} \quad (5)$$

## 9.4 Rezolvarea coliziunilor

Când se detectează o coliziune, camera este împinsă afară din sfera penguinului:

```
1 void resolveCameraCollision(CameraCollider& cam, const Penguin& penguin)
2 {
3     // Vector de la penguin la camera
4     glm::vec3 direction = cam.position - penguin.position;
5     float distance = glm::length(direction);
```

```

5
6 // Calculare overlap (cat se suprapun sferele)
7 float overlap = (cam.radius + penguin.radius) - distance;
8
9 if (overlap > 0.0f) { // Exista coliziune
10     if (distance > 0.0f) { // Camera nu e exact la pozitia
11         penguinului
12         // Normalizeaza directia
13         glm::vec3 pushDirection = glm::normalize(direction);
14
15         // Impinge camera afara din collider
16         glm::vec3 correction = pushDirection * overlap;
17         cam.position += correction;
18     }
19     else {
20         // Camera exact la pozitia penguinului
21         // Impinge pe axa X
22         cam.position += glm::vec3(cam.radius + penguin.radius, 0.0f,
23         0.0f);
24     }
25 }
26 }

```

Listing 52: Funcția resolveCameraCollision()

#### Explicație algoritm:

1. Se calculează distanța actuală între centre
2. Se calculează overlap-ul (cât de mult se suprapun sferele)
3. Se calculează direcția de separare (normalizată)
4. Camera este mutată cu distanța overlap în direcția de separare

## 9.5 Integrare în loop-ul principal

Coliziunile sunt verificate în fiecare frame, dar **doar când animația nu este activă**:

```

1 while (!glfwWindowShouldClose(myWindow.getWindow())) {
2     float currentFrame = glfwGetTime();
3     float deltaTime = currentFrame - lastFrame;
4     lastFrame = currentFrame;
5
6     if (isPlayingAnimation) {
7         updateCameraAnimation(currentFrame);
8     }
9     else {
10        processMovement();
11
12        // Actualizare pozitie collider camera
13        cameraCollider.position = myCamera.getCameraPosition();
14
15        // Verificare coliziuni cu toti penguinii
16        for (auto& penguin : penguins) {
17            if (checkCameraCollision(cameraCollider, penguin)) {
18                // Rezolvare coliziune
19                resolveCameraCollision(cameraCollider, penguin);

```

```

20         // Actualizare pozitie camera
21         myCamera.setCameraPosition(cameraCollider.position);
22     }
23 }
24 }
25
26 updateParticles(deltaTime, firePos);
27 renderScene();
28 renderParticles(fireShader, firePos, particleVAO);
29
30 glfwPollEvents();
31 glfwSwapBuffers(myWindow.getWindow());
32 }

```

Listing 53: Verificare coliziuni în main loop

## 9.6 Parametri Sistem Coliziuni

| Parametru        | Valoare      | Descriere                |
|------------------|--------------|--------------------------|
| Raza cameră      | 2.0          | Unități în world space   |
| Raza penguin     | 90.0         | Unități în world space   |
| Număr pinguini   | 9            | Total collidere active   |
| Verificări/frame | 9            | O verificare per penguin |
| Tip collider     | Sphere       | Simplu și eficient       |
| Rezolvare        | Displacement | Împingere cameră         |

Tabela 6: Parametri sistem coliziuni

## 9.7 Avantaje Sphere Colliders

- **Simplu:** O singură verificare de distanță
- **Rapid:** Complexitate  $O(1)$  per coliziune
- **Scalabil:** Verificare cu toți pinguinii =  $O(n)$
- **Smooth:** Rezolvare graduală, fără blocaje

# 10 Sistemul de particule pentru implementarea focului

Proiectul implementează un sistem avansat de particule pentru simularea focului, folosind **instanced rendering** și **billboard particles**.

## 10.1 Structura particulei

```

1 struct Particle {
2     glm::vec3 pos;           // Pozitie in world space
3     glm::vec3 velocity;     // Viteza de miscare
4     glm::vec4 color;        // Culoare RGBA

```

```

5     float size;           // Dimensiune (world units)
6     float life;          // Viata ramasa (secunde)
7     float maxLife;       // Viata maxima
8     bool isFire;         // True = foc, False = fum
9 };
10
11 const int MAX_PARTICLES = 1000;
12 Particle particles[MAX_PARTICLES];

```

Listing 54: Structura Particle

## 10.2 Inițializarea buffer-elor OpenGL

Sistemul folosește **instanced rendering** cu un quad partajat și date per-instanță:

```

1 GLuint particleVAO, quadVBO, quadEBO, instanceVBO;
2
3 // Quad vertices (acelasi pentru toate particulele)
4 GLfloat quadVertices[] = {
5     -1.0f, -1.0f, 0.0f, // bottom-left
6     1.0f, -1.0f, 0.0f, // bottom-right
7     1.0f, 1.0f, 0.0f, // top-right
8     -1.0f, 1.0f, 0.0f // top-left
9 };
10
11 GLuint quadIndices[] = {
12     0, 1, 2, // primul triunghi
13     2, 3, 0 // al doilea triunghi
14 };
15
16 // Setup VAO
17 glGenVertexArrays(1, &particleVAO);
18 glBindVertexArray(particleVAO);
19
20 // Quad vertices (shared)
21 glGenBuffers(1, &quadVBO);
22 glBindBuffer(GL_ARRAY_BUFFER, quadVBO);
23 glBufferData(GL_ARRAY_BUFFER, sizeof(quadVertices),
24             quadVertices, GL_STATIC_DRAW);
25 glEnableVertexAttribArray(0);
26 glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE,
27                       3 * sizeof(float), (void*)0);
28
29 // Indices
30 glGenBuffers(1, &quadEBO);
31 glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, quadEBO);
32 glBufferData(GL_ELEMENT_ARRAY_BUFFER, sizeof(quadIndices),
33             quadIndices, GL_STATIC_DRAW);
34
35 // Instance data (diferit pentru fiecare particula)
36 glGenBuffers(1, &instanceVBO);
37 glBindBuffer(GL_ARRAY_BUFFER, instanceVBO);
38 glBufferData(GL_ARRAY_BUFFER, MAX_PARTICLES * 9 * sizeof(float),
39             nullptr, GL_DYNAMIC_DRAW);
40
41 // Layout: pos(3) + color(4) + size(1) + life(1) = 9 floats
42
43 // Position

```



```

44 glEnableVertexArray(1);
45 glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, 9 * sizeof(float),
46                         (void*)0);
47 glVertexAttribDivisor(1, 1); // o instanta per particula
48
49 // Color
50 glEnableVertexArray(2);
51 glVertexAttribPointer(2, 4, GL_FLOAT, GL_FALSE, 9 * sizeof(float),
52                         (void*)(3 * sizeof(float)));
53 glVertexAttribDivisor(2, 1);
54
55 // Size
56 glEnableVertexArray(3);
57 glVertexAttribPointer(3, 1, GL_FLOAT, GL_FALSE, 9 * sizeof(float),
58                         (void*)(7 * sizeof(float)));
59 glVertexAttribDivisor(3, 1);
60
61 // Life
62 glEnableVertexArray(4);
63 glVertexAttribPointer(4, 1, GL_FLOAT, GL_FALSE, 9 * sizeof(float),
64                         (void*)(8 * sizeof(float)));
65 glVertexAttribDivisor(4, 1);

```

Listing 55: Configurare VAO și VBO pentru particule

## 10.3 Generarea particulelor

### 10.3.1 Funcția respawnParticle

Fiecare particulă este regenerată cu proprietăți aleatorii:

```

1 void respawnParticle(Particle& particle, glm::vec3 firePos) {
2     // Decide daca e foc sau fum (80% foc, 20% fum)
3     particle.isFire = ((float)rand() / RAND_MAX) < 0.8f;
4
5     float angle = ((float)rand() / RAND_MAX) * 2.0f * 3.14159f;
6
7     if (particle.isFire) {
8         // FOC - porneste din/aproape de baza focului
9         float radius = sqrt(((float)rand() / RAND_MAX) * 85.0f);
10        float yOffset = ((float)rand() / RAND_MAX) * 150.0f;
11
12        particle.pos = firePos + glm::vec3(
13            radius * cos(angle),
14            yOffset,
15            radius * sin(angle)
16        );
17
18        // Viteza sus cu variatie laterala
19        particle.velocity = glm::vec3(
20            (((float)rand() / RAND_MAX) - 0.5f) * 4.0f, // x
21            15.0f + ((float)rand() / RAND_MAX) * 10.0f, // y (sus)
22            (((float)rand() / RAND_MAX) - 0.5f) * 4.0f // z
23        );
24
25        // Culoare bazata pe distanta de la centru
26        float distanceRatio = radius / 150.0f;
27        if (distanceRatio < 0.1f) {

```

```

28         // CENTRU - GALBEN
29         particle.color = glm::vec4(1.0f, 1.0f, 0.4f, 1.0f);
30     }
31     else if (distanceRatio < 0.65f) {
32         // MIJLOC - PORTOCALIU
33         particle.color = glm::vec4(1.0f, 0.5f, 0.1f, 0.9f);
34     }
35     else {
36         // MARGINE - ROSU
37         particle.color = glm::vec4(0.95f, 0.2f, 0.05f, 0.8f);
38     }
39
40     particle.size = 20.0f + ((float)rand() / RAND_MAX) * 30.0f;
41     particle.maxLife = 0.5f + ((float)rand() / RAND_MAX) * 0.4f;
42 }
43 else {
44     // FUM - porneste mai sus
45     float radius = sqrt((float)rand() / RAND_MAX) * 150.0f;
46     float yOffset = 200 + ((float)rand() / RAND_MAX) * 1000.0f;
47
48     particle.pos = firePos + glm::vec3(
49         radius * cos(angle),
50         yOffset,
51         radius * sin(angle)
52     );
53
54     particle.velocity = glm::vec3(
55         (((float)rand() / RAND_MAX) - 0.5f) * 10.0f,
56         8.0f + ((float)rand() / RAND_MAX) * 25.0f,
57         (((float)rand() / RAND_MAX) - 0.5f) * 12.0f
58     );
59
60     // Culoare gri cu alpha redus
61     float darkness = 0.1f + ((float)rand() / RAND_MAX) * 0.2f;
62     particle.color = glm::vec4(darkness, darkness, darkness, 0.3f);
63
64     particle.size = 70.0f + ((float)rand() / RAND_MAX) * 70.0f;
65     particle.maxLife = 2.5f + ((float)rand() / RAND_MAX) * 1.5f;
66 }
67
68 particle.life = particle.maxLife;
69 }

```

Listing 56: Respawn particule - foc și fum

## 10.4 Actualizare particule

Particulele sunt actualizate în fiecare frame cu fizică și tranziții de culoare:

```

1 void updateParticles(float deltaTime, glm::vec3 firePos) {
2     int newParticles = (int)(deltaTime * 200.0f); // 200 particule/sec
3     int particlesSpawned = 0;
4
5     for (int i = 0; i < MAX_PARTICLES; i++) {
6         Particle& p = particles[i];
7
8         if (p.life > 0.0f) {
9             // Decrementare viata

```

```

10     p.life -= deltaTime;
11
12     // Update pozitie
13     p.pos += p.velocity * deltaTime;
14
15     if (p.isFire) {
16         // FOC - fizica
17         p.velocity.x += ((float)rand() / RAND_MAX - 0.5f)
18                         * 10.0f * deltaTime;
19         p.velocity.z += ((float)rand() / RAND_MAX - 0.5f)
20                         * 10.0f * deltaTime;
21         p.velocity.y -= 3.0f * deltaTime; // gravitate
22
23         // Tranzitii culoare: galben -> portocaliu -> rosu
24         float lifeRatio = p.life / p.maxLife;
25         if (lifeRatio > 0.7f) {
26             p.color = glm::vec4(1.0f, 1.0f, 0.4f, 1.0f);
27         }
28         else if (lifeRatio > 0.4f) {
29             p.color.r = 1.0f;
30             p.color.g = 0.5f + (lifeRatio - 0.4f) * 1.67f;
31             p.color.b = 0.1f;
32             p.color.a = 0.9f;
33         }
34         else {
35             p.color.r = 1.0f;
36             p.color.g = 0.2f * (lifeRatio / 0.4f);
37             p.color.b = 0.0f;
38             p.color.a = lifeRatio * 2.0f; // fade out
39         }
40     }
41     else {
42         // FUM - se extinde si urca
43         p.velocity.x *= 0.97f; // friction
44         p.velocity.z *= 0.97f;
45         p.velocity.y += 2.0f * deltaTime;
46
47         p.size += 60.0f * deltaTime; // expansiune
48
49         float lifeRatio = p.life / p.maxLife;
50         float brightness = 0.4f + ((1.0f - lifeRatio) * 0.3f);
51         p.color.r = brightness;
52         p.color.g = brightness;
53         p.color.b = brightness;
54         p.color.a = lifeRatio * 0.3f; // fade out
55     }
56 }
57 else {
58     // Respawn particula moarta
59     if (particlesSpawned < newParticles) {
60         respawnParticle(p, firePos);
61         particlesSpawned++;
62     }
63 }
64 }
65 }

```

Listing 57: Funcția updateParticles()

## 10.5 Vertex Shader - Billboard Rendering

Particulele sunt orientate automat spre cameră (billboarding):

```
1 #version 410 core
2
3 layout(location = 0) in vec3 vertexPosition; // Quad vertices
4 layout(location = 1) in vec3 particlePos; // Instance data
5 layout(location = 2) in vec4 particleColor;
6 layout(location = 3) in float particleSize;
7 layout(location = 4) in float particleLife;
8
9 out vec4 color;
10 out float life;
11 out vec2 uv;
12
13 uniform mat4 view;
14 uniform mat4 projection;
15
16 void main()
17 {
18     // Extrage vectorii right si up din view matrix
19     vec3 cameraRight = vec3(view[0][0], view[1][0], view[2][0]);
20     vec3 cameraUp = vec3(view[0][1], view[1][1], view[2][1]);
21
22     // Construiește pozitia worldPos orientata spre camera
23     vec3 worldPos = particlePos
24         + cameraRight * vertexPosition.x * particleSize
25         + cameraUp * vertexPosition.y * particleSize;
26
27     gl_Position = projection * view * vec4(worldPos, 1.0);
28
29     // Pass data catre fragment shader
30     color = particleColor;
31     life = particleLife;
32     uv = vertexPosition.xy * 0.5 + 0.5; // [-1,1] -> [0,1]
33 }
```

Listing 58: fire\_instanced.vert

## 10.6 Fragment Shader - Gradient circular

Particulele au un gradient radial pentru aspect soft:

```
1 #version 410 core
2
3 in vec4 color;
4 in float life;
5 in vec2 uv;
6
7 out vec4 fragmentColour;
8
9 void main()
10 {
11     // Coordonate centrate (-0.5 la 0.5)
12     vec2 coord = uv - vec2(0.5);
13     float dist = length(coord);
14 }
```

```

15 // Discard fragmente in afara cercului
16 if(dist > 0.5)
17     discard;
18
19 // Gradient radial: centru (1.0) -> margine (0.0)
20 float alpha = 1.0 - (dist * 2.0);
21
22 // Smoother falloff pentru aspect realistic
23 alpha = pow(alpha, 2.0);
24
25 // Fade-out pe durata vietii
26 alpha *= life;
27
28 // Multiplica cu alpha din culoare
29 alpha *= color.a;
30
31 fragmentColour = vec4(color.rgb, alpha);
32 }

```

Listing 59: fire\_instanced.frag

## 10.7 Depth Sorting pentru transparență

Particulele sunt sortate de la cele mai îndepărtate la cele mai apropiate (painter's algorithm):

```

1 struct ParticleDistance {
2     int index;
3     float distance;
4 };
5
6 std::vector<ParticleDistance> distances;
7 glm::vec3 cameraPos = myCamera.getCameraPosition();
8
9 // Colecteaza particulele vii si distantele
10 for (int i = 0; i < MAX_PARTICLES; i++) {
11     if (particles[i].life > 0.0f) {
12         float dist = glm::length(cameraPos - particles[i].pos);
13         distances.push_back({ i, dist });
14     }
15 }
16
17 // Sortare: cel mai departe primul
18 std::sort(distances.begin(), distances.end(),
19     [](const ParticleDistance& a, const ParticleDistance& b) {
20         return a.distance > b.distance;
21     });

```

Listing 60: Sortare particule înainte de rendering

## 10.8 Randare particule

```

1 void renderParticles(gps::Shader& shader, glm::vec3 firePos,
2                     GLuint particleVAO) {
3     glEnable(GL_BLEND);
4     glBlendFunc(GL_SRC_ALPHA, GL_ONE); // Additive blending

```

```

5   glDepthMask(GL_FALSE); // Nu scrie in depth buffer
6   glDisable(GL_CULL_FACE);
7
8   shader.useShaderProgram();
9
10  // Upload uniforms
11  glUniformMatrix4fv(viewLoc, 1, GL_FALSE, glm::value_ptr(view));
12  glUniformMatrix4fv(projLoc, 1, GL_FALSE, glm::value_ptr(projection))
13  ;
14
15  // [sortare particule - vezi mai sus]
16
17  // Construiesc instance data
18  std::vector<float> instanceData;
19  for (const auto& pd : distances) {
20      int i = pd.index;
21      const Particle& p = particles[i];
22
23      instanceData.push_back(p.pos.x);
24      instanceData.push_back(p.pos.y);
25      instanceData.push_back(p.pos.z);
26      instanceData.push_back(p.color.r);
27      instanceData.push_back(p.color.g);
28      instanceData.push_back(p.color.b);
29      instanceData.push_back(p.color.a);
30      instanceData.push_back(p.size);
31      instanceData.push_back(p.life / p.maxLife);
32  }
33
34  // Upload instance data
35  glBindVertexArray(particleVAO);
36  glBindBuffer(GL_ARRAY_BUFFER, particleVBO);
37  glBufferSubData(GL_ARRAY_BUFFER, 0,
38      instanceData.size() * sizeof(float),
39      instanceData.data());
40
41  // Instanced rendering
42  int numParticles = instanceData.size() / 9;
43  glDrawElementsInstanced(GL_TRIANGLES, 6, GL_UNSIGNED_INT,
44      0, numParticles);
45
46  // Cleanup
47  glDepthMask(GL_TRUE);
48  glDisable(GL_BLEND);
49  glEnable(GL_CULL_FACE);
50 }

```

Listing 61: Funcția renderParticles()

## 10.9 Parametri sistem particule

| Parametru     | Valoare  | Descriere                 |
|---------------|----------|---------------------------|
| Max particule | 1000     | Capacitate totală         |
| Spawn rate    | 200/sec  | Particule noi per secundă |
| Ratio foc/fum | 80/20    | Procent tip particule     |
| Viață foc     | 0.5-0.9s | Durată particule foc      |
| Viață fum     | 2.5-4.0s | Durată particule fum      |
| Size foc      | 20-50    | World units               |
| Size fum      | 70-140   | World units (expansiune)  |
| Blending      | Additive | GL_SRC_ALPHA + GL_ONE     |
| Sorting       | Depth    | Back-to-front             |

Tabela 7: Parametri sistem particule

## 11 Bibliografie

### 11.1 Resurse pentru vizualizarea scenei

#### [1] Template Laborator - Grafică pe Calculator

- Sursa: Materialele de laborator furnizate de cadrul didactic
- Utilizat pentru: Structura de bază (Window, Camera, Model3D), inițializare OpenGL, error checking

#### [2] LearnOpenGL - Joey de Vries

- URL: <https://learnopengl.com>
- Utilizat pentru: Camera FPS cu mouse/keyboard, matricele MVP, input callbacks

#### [3] GLM (OpenGL Mathematics)

- URL: <https://github.com/g-truc/glm>
- Utilizat pentru: glm::mix (interpolare), glm::normalize, operații cu matrici/-vectori

#### [4] GLFW Documentation

- URL: <https://www.glfw.org/docs/latest/>
- Utilizat pentru: Keyboard/mouse callbacks, cursor modes

#### [5] Fix Your Timestep - Gaffer On Games

- URL: [https://gafferongames.com/post/fix\\_your\\_timestep/](https://gafferongames.com/post/fix_your_timestep/)
- Utilizat pentru: DeltaTime pentru mișcare frame-rate independent

## 11.2 Resurse pentru Iluminare

### [6] LearnOpenGL - Basic Lighting

- URL: <https://learnopengl.com/Lighting/Basic-Lighting>
- Utilizat pentru: Modelul Phong (ambient, diffuse, specular), calculul normalelor în view space

### [7] LearnOpenGL - Light Casters

- URL: <https://learnopengl.com/Lighting/Light-casters>
- Utilizat pentru: Lumini direcționale și punctiforme, formula de atenuare

### [8] OpenGL Shading Language (GLSL) Specification

- URL: [https://www.khronos.org/opengl/wiki/OpenGL\\_Shading\\_Language](https://www.khronos.org/opengl/wiki/OpenGL_Shading_Language)
- Utilizat pentru: Sintaxa shader-elor, funcții built-in (normalize, reflect, dot, pow)

### [9] Real-Time Rendering, 4th Edition

- Autori: Tomas Akenine-Möller, Eric Haines, Naty Hoffman
- Utilizat pentru: Modelul de iluminare Phong, atenuarea luminii punctiforme

## 11.3 Resurse pentru Generarea Umbrelor

### [10] LearnOpenGL - Shadow Mapping

- URL: <https://learnopengl.com/Advanced-Lighting/Shadows/Shadow-Mapping>
- Utilizat pentru: Conceptul de shadow mapping, depth map rendering, light space matrix

### [11] LearnOpenGL - Point Shadows

- URL: <https://learnopengl.com/Advanced-Lighting/Shadows/Point-Shadows>
- Utilizat pentru: Cubemap shadows, atenuare pentru point lights

### [12] Microsoft - Common Techniques to Improve Shadow Depth Maps

- URL: <https://learn.microsoft.com/en-us/windows/win32/dxtecharts/common-techniques-to-improve-shadow-depth-maps>
- Utilizat pentru: PCF (Percentage Closer Filtering), shadow acne prevention, bias calculation

### [13] OpenGL Wiki - Framebuffer Object

- URL: [https://www.khronos.org/opengl/wiki/Framebuffer\\_Object](https://www.khronos.org/opengl/wiki/Framebuffer_Object)
- Utilizat pentru: Configurarea FBO pentru depth texture, attachment setup



## 11.4 Resurse pentru Animarea Componentelor

### [14] LearnOpenGL - Transformations

- URL: <https://learnopengl.com/Getting-started/Transformations>
- Utilizat pentru: Transformări ierarhice, ordinea transformărilor (TRS - Translate, Rotate, Scale)

### [15] GLM Documentation - Transform Functions

- URL: <https://glm.g-truc.net/0.9.9/api/a00247.html>
- Utilizat pentru: glm::translate, glm::rotate, glm::radians

### [16] Skeletal Animation - OGLdev

- URL: <https://ogldev.org/www/tutorial38/tutorial38.html>
- Utilizat pentru: Conceptul de pivot points, transformări relative la părinte

### [17] GLFW Time Input

- URL: [https://www.glfw.org/docs/latest/group\\_\\_input.html#gaa6cf4e7a77158a3b8f](https://www.glfw.org/docs/latest/group__input.html#gaa6cf4e7a77158a3b8f)
- Utilizat pentru: glfwGetTime() pentru animații frame-independent

### [18] Mathematics for 3D Game Programming - Chapter 4

- Autor: Eric Lengyel
- Utilizat pentru: Funcții sinusoidale pentru animații naturale, transformări matriceale ierarhice

## 11.5 Resurse pentru Detectarea și Rezolvarea Coliziunilor

### [19] Real-Time Collision Detection

- Autor: Christer Ericson
- Utilizat pentru: Sphere-sphere collision detection, algoritmul de separare a sferelor

### [20] Game Physics - Collision Detection

- URL: <https://www.toptal.com/game/video-game-physics-part-ii-collision-detect>
- Utilizat pentru: Conceptul de bounding spheres, calculul overlap-ului

### [21] GLM Vector Operations

- URL: <https://glm.g-truc.net/0.9.9/api/a00241.html>
- Utilizat pentru: glm::length() pentru distanță, glm::normalize() pentru direcții